ERPNext v16 Procure-to-Pay (P2P) API Integration Guide

Complete Technical Manual: OAuth2, API Keys, Material Request (PR) & Purchase Order (PO) Lifecycles

Target System: ERPNext v16 / Frappe Framework	**Environment:** Local / Docker (Port 8080)
Document Version: 1.0.0	**Author:** AI Integration Engineering Team

1. Executive Summary & Overview

This technical manual provides a step-by-step guide for authenticating and executing the full **Procure-to-Pay (P2P)** workflow in **ERPNext v16** using Postman or programmatic API clients. It covers setup procedures for prerequisite master data (Items), setup for both **OAuth2 (Password Grant)** and **API Key/Secret Authentication**, and detailed endpoints for managing **Material Requests (Purchase Requisitions - PR)** and **Purchase Orders (PO)**.

2. Prerequisite Setup: Master Data

Before issuing Purchase Requisitions via API, a valid **Item Code** must exist in ERPNext to avoid relational validation errors.

Creating an Item in ERPNext UI

1. Log into the ERPNext Web UI at `http://localhost:8080` as Administrator.
2. In the search bar (Awesomebar), type **Item List** and press `Enter`.
3. Click **+ Add Item** in the top-right corner.
4. Fill in mandatory parameters:
   - **Item Code:** `ITEM-001`
   - **Item Name:** `Test Item`
   - **Item Group:** Select `All Item Groups` or `Products`
   - **Default Unit of Measure (UOM):** `Nos` or `Box`
5. Click **Save**.

3. Authentication Strategies & Setup

Frappe Framework supports two primary machine authentication mechanisms. Choose the strategy best suited for your implementation model:

3.1 Strategy A: Direct API Key & Secret (Recommended for Background Jobs & M2M)

API Key/Secret authentication is non-expiring and connects directly to ERPNext REST resources without session cookie dependencies.

Generating API Keys

1. Go to **User List** → Select your user profile (e.g., `Administrator` or dedicated API User).
2. Scroll down to the **API Access** section.
3. Click **Generate Keys**.
4. Copy and store the generated **API Key** and **API Secret** securely (the Secret is shown only once).

Postman Header Setup for API Keys:

HEADER KEY	HEADER VALUE	EXAMPLE
`Authorization`	`token <API_KEY>:<API_SECRET>`	`token 827abc12345:901def67890`

3.2 Strategy B: OAuth2 Password Grant (Headless Token Generation)

For token-based authorization without interactive browser popups, configure the OAuth Client for Password Grant.

3.2.1 OAuth Client UI Configuration in ERPNext:

FIELD NAME	REQUIRED CONFIGURATION VALUE
App Name (Client Name)	`Procurement Integration App`
Allowed Roles	`Administrator` / `Purchase Manager`
Default Redirect URI	`http://localhost:3000/callback`
Redirect URIs	`http://localhost:3000/callback`
Scopes	`all` `openid` *(Must be entered on separate lines)*
Grant Type	`Authorization Code` & `Password`
Response Type	`Code`
Token Endpoint Auth Method	`Client Secret Basic`

3.2.2 Requesting Access Token via API:

POST `http://localhost:8080/api/method/frappe.integrations.oauth2.get_token`

Headers: `Content-Type: application/x-www-form-urlencoded`

Body (x-www-form-urlencoded):

```
grant_type=password
client_id=<YOUR_CLIENT_ID>
client_secret=<YOUR_CLIENT_SECRET>
username=Administrator
```

```
password=<YOUR_ADMIN_PASSWORD>
scope=all openid
```

Success Response (200 OK):

```
{
    "access_token": "a1b2c3d4e5f6...",
    "expires_in": 3600,
    "token_type": "Bearer",
    "scope": "all openid",
    "refresh_token": "r1s2t3u4v5..."
}
```

Important Note on Frappe OAuth & REST Endpoints

In ERPNext, OAuth2 Bearer tokens work natively with RPC endpoints (`/api/method/...`). When accessing standard resource endpoints (`/api/resource/...`) with Bearer tokens, disable Postman's automatic cookie storage (clear `sid` cookies) or use the RPC methods (`frappe.client.insert`, `frappe.client.submit`) to avoid `AuthenticationError` or `session_expired` errors.

4. Complete Procure-to-Pay (PR to PO) API Lifecycle

Step 1: Create Purchase Requisition (Material Request) - Draft

Creates a new Purchase Requisition in **Draft** status (`docstatus: 0`).

Option A: Direct Resource Endpoint (Using API Key Auth)

POST `http://localhost:8080/api/resource/Material Request`

Option B: RPC Endpoint (Using OAuth2 Bearer Auth)

POST `http://localhost:8080/api/method/frappe.client.insert`

Headers:

`Authorization: Bearer <YOUR_ACCESS_TOKEN>` OR `token <API_KEY>:<API_SECRET>`
`Content-Type: application/json`

Payload (raw JSON for Option B):

```
{
  "doc": {
    "doctype": "Material Request",
    "material_request_type": "Purchase",
    "schedule_date": "2026-09-01",
    "items": [
      {
        "item_code": "ITEM-001",
        "qty": 10,
        "schedule_date": "2026-09-01"
      }
    ]
  }
}
```

Expected Response (200 OK):

```
{
  "message": {
    "name": "MAT-REQ-2026-00001",
    "docstatus": 0,
    "material_request_type": "Purchase",
    "status": "Draft",
    "items": [
      {
        "name": "mat-req-item-001",
        "item_code": "ITEM-001",
        "qty": 10
      }
    ]
  }
}
```

Step 2: Approve & Submit Purchase Requisition

Transitions the document status from **Draft** (0) to **Submitted** (1).

Option A: Direct Resource Endpoint

PUT `http://localhost:8080/api/resource/Material Request/MAT-REQ-2026-00001`

Payload: `{"docstatus": 1}`

Option B: RPC Endpoint (OAuth2 Compatible)

POST `http://localhost:8080/api/method/frappe.client.submit`

Payload:

```
{
  "doc": {
    "doctype": "Material Request",
    "name": "MAT-REQ-2026-00001"
  }
}
```

Step 3: Convert Approved PR into a Purchase Order (PO)

Invokes ERPNext's document mapper function to automatically construct a mapped Purchase Order JSON structure from the approved PR.

POST `http://localhost:8080/api/method/erpnext.buying.doctype.material_request.material_request.make_purchase_order`

Headers: `Content-Type: application/json`

Payload:

```
{
  "source_name": "MAT-REQ-2026-00001"
}
```

Response: Returns the complete mapped Purchase Order object pre-filled with items, quantities, and reference links back to `MAT-REQ-2026-00001`.

Step 4: Save & Submit Purchase Order

Submits the mapped Purchase Order into ERPNext.

POST `http://localhost:8080/api/method/frappe.client.insert`

Payload:

```
{
  "doc": {
    "doctype": "Purchase Order",
    "supplier": "Your-Supplier-Name",
    "schedule_date": "2026-09-01",
    "items": [
      {
        "material_request": "MAT-REQ-2026-00001",
        "item_code": "ITEM-001",
        "qty": 10,
        "rate": 100,
        "schedule_date": "2026-09-01"
      }
    ]
  }
}
```

5. Document Deletion & Cancellation Rules

ERPNext maintains strict database transactional integrity. You **cannot delete a submitted document directly**.

TARGET STATE	PREREQUISITE STATUS	HTTP METHOD & ENDPOINT	PAYLOAD / ACTION
Delete Draft PR / PO	`docstatus: 0` (Draft)	DELETE `/api/resource/Material Request/{ID}`	None (Direct Deletion)
Cancel Submitted PR / PO	`docstatus: 1` (Submitted)	PUT `/api/resource/Material Request/{ID}`	`{"docstatus": 2}`
Delete Cancelled PR / PO	`docstatus: 2` (Cancelled)	DELETE `/api/resource/Material Request/{ID}`	None (Permanent Purge)

6. API Endpoint Quick Reference Table

ACTION	HTTP METHOD	ENDPOINT URL	REQUIRED KEY PAYLOAD FIELDS
Get Access Token	POST	`/api/method/frappe.integrations.oauth2.get_token`	`grant_type`, `client_id`, `username`, `password`
Create PR (Draft)	POST	`/api/method/frappe.client.insert`	`doctype`, `material_request_type`, `items`

ACTION	HTTP METHOD	ENDPOINT URL	REQUIRED KEY PAYLOAD FIELDS
Submit PR	POST	`/api/method/frappe.client.submit`	`doctype`, `name`
Map PR to PO	POST	`/api/method/...make_purchase_order`	`source_name`
Cancel Doc	PUT	`/api/resource/Material Request/{ID}`	`{"docstatus": 2}`
Delete Doc	DELETE	`/api/resource/Material Request/{ID}`	None